

Prompt:

8/17/2026, 10:19:51 PM

lanjut Tahap AE

Response:

8/17/2026, 10:19:51 PM

Remembering

Reviewed and preserved your Tahap AE trading architecture and validation integrity

Worked for 4s

Tahap AE — Probability Calibration, Statistical Validation & Backtest Integrity

Baik. Setelah Tahap AD, kita sudah memiliki **trade engine** yang dapat menghasilkan keputusan seperti

$$\theta_t^* = (Entry, SL, TP, Q, TTL)^*$$

berdasarkan estimasi:

$$\hat{\mu}_t, \quad \hat{\sigma}_t, \quad P(TP), \quad P(SL), \quad P(TimeStop),$$

dan expected value.

Tetapi sekarang muncul pertanyaan yang jauh lebih fundamental:

Seberapa besar kita boleh mempercayai probabilitas dan expected value yang dihasilkan model?

Kalau model mengatakan

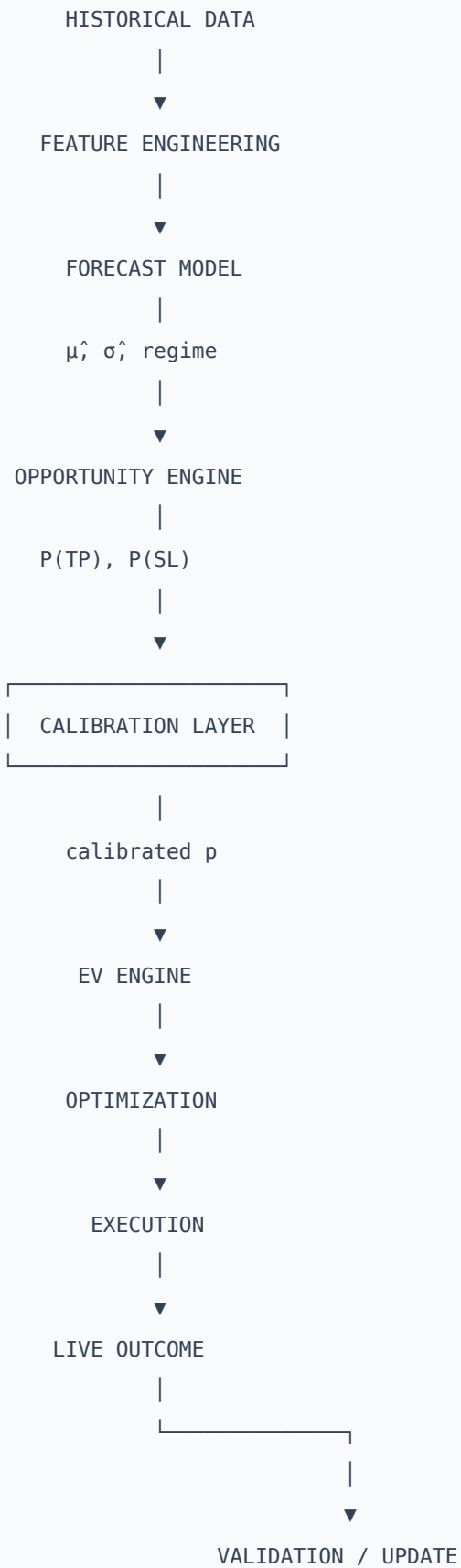
$$P(TP) = 0.80,$$

tetapi pada kenyataannya hanya 55% trade yang berhasil, maka optimasi matematis kita akan mengoptimalkan **dunia yang salah**.

Jadi Tahap AE kita jadikan sebagai **lapisan validasi statistik yang berdiri di antara model prediksi dan optimizer**.

AE.1. Prinsip utama

Arsitektur kita sekarang seharusnya menjadi:



Dengan demikian:

Prediction $\neq$ Probability $\neq$ Decision.

Ketiganya harus dipisahkan.

AE.2. Prediction versus probability

Misalkan model memberikan:

$$\hat{p}_t = 0.72.$$

Artinya secara ideal:

Pada kumpulan kondisi yang mempunyai probabilitas prediksi sekitar 72%, sekitar 72% outcome seharusnya benar-benar terjadi.

Ini adalah konsep **calibration**.

Jika model menghasilkan:

$$\hat{p} = 0.7$$

untuk 1.000 trade, kita mengharapkan kira-kira:

$$700$$

trade berhasil.

Bukan berarti setiap kelompok kecil harus tepat 70%, tetapi secara statistik harus mendekati.

AE.3. Perfect calibration

Secara formal, model dikatakan calibrated apabila:

$P(Y = 1 \mid \hat{p} = p) = p.$

Dengan:

$$Y = \begin{cases} 1, & TP \text{ tercapai terlebih dahulu} \\ 0, & SL \text{ tercapai terlebih dahulu.} \end{cases}$$

Jadi:

$$P(TP \mid \hat{p} = p) = p.$$

Ini sangat penting.

AE.4. Contoh model yang tidak calibrated

Misalkan kita mengelompokkan prediction:

Prediksi	Jumlah trade	TP aktual
0.50	100	48
0.60	100	51
0.70	100	55
0.80	100	61
0.90	100	68

Model jelas **overconfident**.

Model mengatakan:

90%

tetapi kenyataannya:

68%.

Jika optimizer menggunakan 90%, EV akan sangat overestimated.

AE.5. Reliability diagram

Cara visual paling sederhana:

$$x = \hat{p}$$

dan:

y = actual frequency.

Perfect calibration adalah:

$$y = x.$$

Misalnya:



Semakin dekat terhadap diagonal, semakin calibrated.

AE.6. Brier Score

Untuk binary outcome, kita dapat menggunakan:

$$BS = \frac{1}{N} \sum_{i=1}^N (\hat{p}_i - y_i)^2.$$

Dengan:

$$0 \leq BS \leq 1.$$

Semakin kecil:

$$BS \downarrow \Rightarrow \text{lebih baik.}$$

AE.7. Mengapa Brier Score sangat cocok untuk sistem kita?

Karena kita tidak hanya peduli:

apakah trade benar?

Kita peduli:

apakah probabilitas yang diberikan model masuk akal?

Misalkan:

Model A

$$p = 0.51$$

ketika:

$$y = 1.$$

Error:

$$(0.51 - 1)^2 = 0.2401.$$

Model B

$$p = 0.95$$

ketika:

$$y = 0.$$

Error:

$$(0.95 - 0)^2 = 0.9025.$$

Model B mendapat hukuman jauh lebih besar karena **sangat percaya diri tetapi salah**.

Ini persis jenis kesalahan yang berbahaya bagi optimizer kita.

AE.8. Log-loss

Kita juga menggunakan:

$$LL = -\frac{1}{N} \sum_{i=1}^N [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)].$$

Jika:

$$y = 1$$

dan:

$$p \rightarrow 0,$$

maka:

$$-\ln p \rightarrow \infty.$$

Artinya prediksi yang **sangat yakin tetapi salah** mendapat hukuman sangat besar.

AE.9. Brier versus Log-loss

Keduanya mengukur kualitas probabilitas, tetapi sifatnya berbeda.

Metric	Fokus
Accuracy	klasifikasi
Brier Score	calibration + probabilistic accuracy
Log-loss	penalti confidence yang salah
Sharpe	risk-adjusted return
PnL	profitability

Kita tidak boleh menggunakan Sharpe untuk menggantikan calibration.

AE.10. Calibration tidak sama dengan profitability

Ini sangat penting.

Model dapat memiliki:

$$BS$$

yang sangat bagus tetapi tidak profitable setelah:

- fee,
- slippage,
- market impact,
- spread.

Sebaliknya model dapat profitable tetapi poorly calibrated.

Jadi kita membutuhkan **dua dimensi evaluasi**:

Statistical Quality

dan:

Economic Quality.

AE.11. Economic validity

Untuk setiap trade:

$$EV_i = p_i G_i - (1 - p_i) L_i - C_i.$$

Tetapi outcome aktual:

$$R_i.$$

Kita ingin:

$$E[R_i \mid EV_i > 0] > 0.$$

Ini adalah pengujian apakah probabilistic forecast benar-benar menghasilkan **economic value**.

AE.12. Jangan melakukan random train-test split

Ini adalah salah satu aturan terpenting.

Misalnya data:

2021 ————— 2025

Jangan:

Random 80% → Train

Random 20% → Test

Karena data finansial adalah **time dependent**.

Random split dapat menyebabkan informasi masa depan masuk ke training.

AE.13. Time-series split

Kita gunakan:

Train	Test
2021 — 2022	2023

kemudian:

Train	Test
2021 — 2023	2024

kemudian:

Train	Test
2021 — 2024	2025

Ini disebut:

Walk-Forward Validation.

AE.14. Mengapa walk-forward?

Karena situasi live sebenarnya adalah:

$$\boxed{Past \rightarrow Model \rightarrow Future.}$$

Bukan:

$$Future \rightarrow Model \rightarrow Past.$$

Walk-forward mempertahankan arah waktu.

AE.15. Expanding window

Salah satu bentuk:

$$Train_1 = [t_0, t_1]$$

$$Test_1 = [t_1, t_2].$$

Kemudian:

$$Train_2 = [t_0, t_2]$$

$$Test_2 = [t_2, t_3].$$

Dan:

$$Train_3 = [t_0, t_3]$$

$$Test_3 = [t_3, t_4].$$

Ini disebut:

$$\boxed{Expanding Window.}$$

AE.16. Rolling window

Alternatif:

$$Train_1 = [t_0, t_1]$$

$$Test_1 = [t_1, t_2].$$

Kemudian:

$$Train_2 = [t_1, t_2]$$

$$Test_2 = [t_2, t_3].$$

Dan seterusnya.

Ini:

Rolling Window.

Untuk pasar yang mengalami structural regime changes, rolling window sering sangat menarik karena model tidak terlalu bergantung pada sejarah yang terlalu lama.

AE.17. Purged cross-validation

Ada masalah yang lebih halus.

Misalkan kita membuat label berdasarkan:

$$t \rightarrow t + 60 \text{ menit.}$$

Training sample pada:

$$t = 10 : 00$$

menggunakan informasi sampai:

$$11 : 00.$$

Sementara test dimulai:

10 : 30.

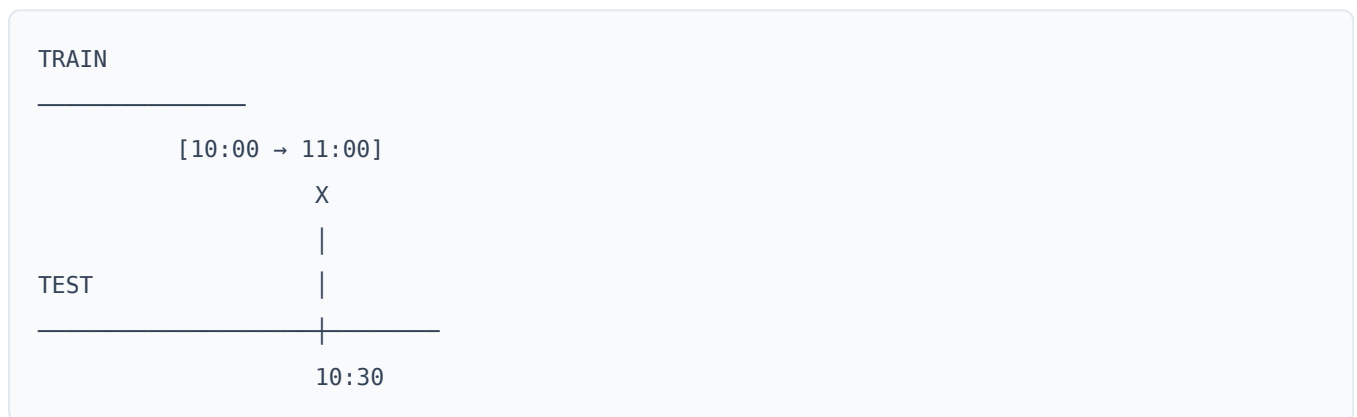
Maka dua sample tersebut overlap.

Itu adalah leakage.

AE.18. Purgings

Kita menghapus training observations yang labelnya overlap dengan test interval.

Misalnya:



Sample training tersebut harus dibuang.

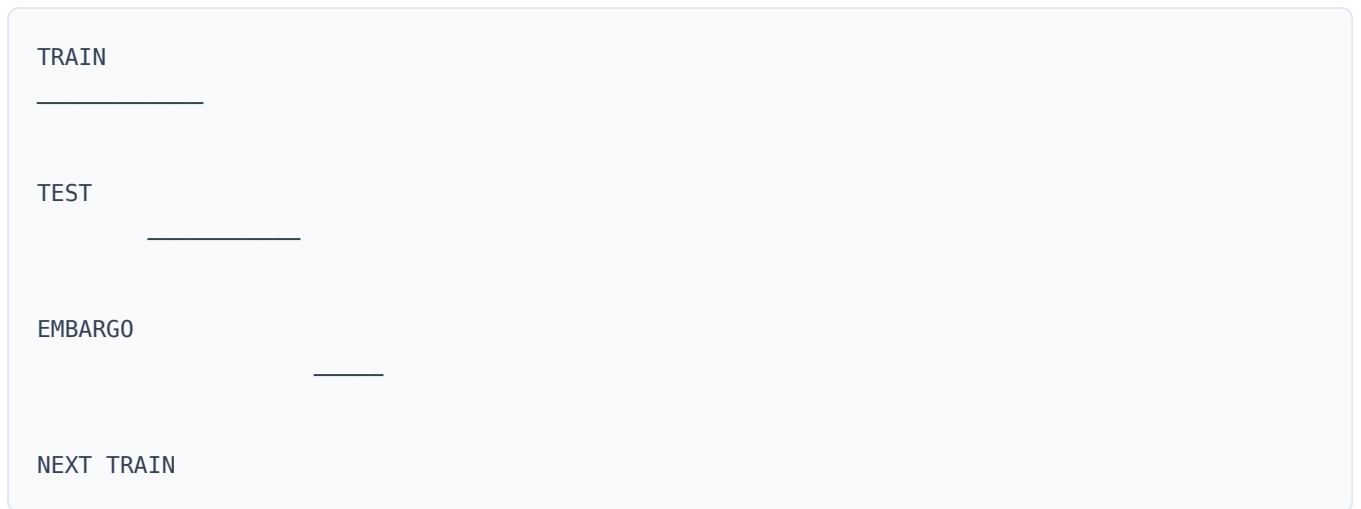
Ini disebut:

Purgings.

AE.19. Embargo

Masih ada masalah lain.

Setelah test period, kita dapat memberikan gap:



Gap ini mencegah ketergantungan temporal merembes antar-fold.

Disebut:

Embargo.

AE.20. Purged + Embargoed CV

Jadi validasi kita idealnya:

Purged Walk-Forward Validation

dengan:

Embargo.

Ini jauh lebih tepat untuk strategi yang memiliki:

- overlapping labels,
- holding period,
- multiple horizons,
- triple-barrier outcomes.

AE.21. Triple-barrier labeling

Ini sangat cocok dengan Tahap AD.

Untuk setiap entry:

$$t_i,$$

kita menentukan:

$$TP_i, \quad SL_i, \quad T_{max,i}.$$

Kemudian label:

$$Y_i = \begin{cases} +1, & TP \text{ tercapai dahulu} \\ -1, & SL \text{ tercapai dahulu} \\ 0, & TimeStop \text{ tercapai dahulu.} \end{cases}$$

Jadi kita tidak menggunakan label:

$$P_{t+1} > P_t.$$

Melainkan outcome yang benar-benar berkaitan dengan trade.

AE.22. Ini membuat training dan validation konsisten

Tahap AD:

$$P(TP), P(SL), P(TimeStop)$$

Tahap AE:

$$Y \in \{TP, SL, TimeStop\}.$$

Dengan demikian probabilistic model memprediksi **event yang sama dengan event yang digunakan optimizer**.

Ini sangat penting.

AE.23. Multiclass calibration

Sekarang probabilitas kita bukan hanya binary.

Model menghasilkan:

$$\mathbf{p}_i = \begin{bmatrix} p_{TP} \\ p_{SL} \\ p_{TS} \end{bmatrix}$$

dengan:

$$p_{TP} + p_{SL} + p_{TS} = 1.$$

Maka kita dapat mengevaluasi calibration masing-masing outcome.

Misalnya:

$$P(TP) = 0.70.$$

Di antara semua trade dengan prediksi sekitar 70%, actual TP harus mendekati 70%.

AE.24. Calibration map

Jika model ternyata overconfident, kita dapat membangun:

$$p_{cal} = g(p_{raw}).$$

Misalnya:

$$p_{raw} = 0.80$$

tetapi calibration menunjukkan:

$$p_{cal} = 0.68.$$

Optimizer **harus menggunakan**:

$$0.68,$$

bukan:

$$0.80.$$

AE.25. Platt scaling

Untuk binary model, salah satu metode:

$$p_{cal} = \frac{1}{1 + \exp(Af + B)}$$

dengan f adalah raw model score.

Parameter:

$$A, B$$

dipelajari menggunakan validation data.

AE.26. Isotonic regression

Alternatif yang lebih fleksibel:

$$p_{cal} = g(p_{raw})$$

dengan constraint:

$$p_1 \leq p_2 \Rightarrow g(p_1) \leq g(p_2).$$

Artinya calibration function monotonic.

Ini berguna ketika hubungan antara raw probability dan actual frequency tidak linear.

AE.27. Jangan calibration menggunakan test set

Ini sangat penting.

Urutannya:

TRAIN



Model



Calibration set



Calibrator



FINAL TEST

Kalau test digunakan untuk fitting calibration:

test set tidak lagi benar-benar out-of-sample.

AE.28. Confidence interval

Misalkan:

$$\hat{p} = 0.60$$

dari:

$$N = 100.$$

Jangan langsung berkata:

$$p = 60\%.$$

Kita perlu uncertainty.

Untuk binary proportion:

$$\hat{p} = \frac{k}{N}.$$

Standard error:

$$SE(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{N}}.$$

AE.29. Contoh

Jika:

$$\hat{p} = 0.60, \quad N = 100,$$

maka:

$$SE = \sqrt{\frac{0.6(0.4)}{100}}$$

$$= 0.049.$$

Jadi uncertainty sekitar:

$$4.9\%.$$

Kita tidak boleh memperlakukan 60% sebagai angka eksak.

AE.30. Wilson interval

Untuk probability yang lebih robust daripada Wald interval, kita dapat menggunakan Wilson interval:

$$\frac{\hat{p} + \frac{z^2}{2N} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}}.$$

Untuk confidence level 95%:

$$z \approx 1.96.$$

AE.31. Mengapa uncertainty penting untuk EV?

Misalkan:

$$p = 0.55.$$

Dengan:

$$R_{TP} = 2R, \quad R_{SL} = 1R.$$

Maka:

$$EV = 0.55(2) - 0.45(1) = 0.65R.$$

Terlihat bagus.

Tetapi jika actual probability uncertainty sangat besar, misalnya:

$$p \in [0.45, 0.65],$$

maka:

$$EV_{low} = 0.45(2) - 0.55 = 0.35R.$$

Masih positif.

Tetapi untuk setup lain mungkin:

$$EV$$

berubah dari positif menjadi negatif.

AE.32. Conservative EV

Kita dapat menggunakan lower confidence bound:

$$\boxed{p_L}$$

daripada:

$$\hat{p}.$$

Kemudian:

$$\boxed{EV_{conservative} = p_L G - (1 - p_L)L - C.}$$

Trade hanya dipertimbangkan jika:

$$EV_{conservative} > 0.$$

Ini membuat sistem lebih defensif.

AE.33. Bootstrap

Kita juga perlu mengukur uncertainty terhadap:

- mean return,
- Sharpe,
- max drawdown,
- win rate,
- EV.

Bootstrap dilakukan dengan resampling data.

Misalnya:

```
Original trades
1 2 3 4 5 ... N

Bootstrap 1
3 3 8 1 7 ... N

Bootstrap 2
2 9 9 4 1 ... N

...
```

Kemudian hitung metric untuk setiap sample.

AE.34. Distribusi Sharpe

Misalkan:

$$SR_1, SR_2, \dots, SR_B$$

dari B bootstrap samples.

Kita memperoleh distribusi:

$$f(SR).$$

Kemudian kita dapat mengambil:

$$CI_{95\%} = [SR_{2.5\%}, SR_{97.5\%}].$$

AE.35. Tetapi bootstrap biasa punya masalah

Return finansial memiliki:

- autocorrelation,
- volatility clustering,
- regime dependence.

Random resampling individual trade dapat menghancurkan struktur temporal.

Karena itu untuk beberapa analisis kita dapat menggunakan:

$$\textit{Block Bootstrap}.$$

Kita melakukan resampling blok waktu, bukan observasi individual secara independen.

AE.36. Sharpe Ratio

Untuk return:

$$r_1, \dots, r_N,$$

Sharpe:

$$SR = \frac{\bar{r} - r_f}{s_r}.$$

Untuk short-term trading, r_f sering relatif kecil pada horizon sangat pendek, tetapi jangan mengabaikannya secara konseptual.

AE.37. Masalah Sharpe klasik

Sharpe tinggi tidak selalu berarti strategi bagus.

Misalnya:

$$SR = 3.$$

Tetapi diperoleh dari:

- 1000 konfigurasi strategy,
- 100 parameter combinations,
- berbagai TP,
- berbagai SL,
- berbagai assets.

Kalau kita memilih yang terbaik setelah melihat semua hasil, kita mengalami:

Multiple Testing Bias.

AE.38. Ini sangat berbahaya

Bayangkan kita mencoba:

10,000

strategi random.

Walaupun semua sebenarnya tidak punya edge, sebagian hampir pasti akan terlihat luar biasa.

Ini adalah:

Selection Bias.

atau:

Data Snooping.

AE.39. Contoh

Kita mencoba:

1000

kombinasi:

$TP, SL, lookback, features, model, threshold.$

Strategi terbaik memiliki:

$Sharpe = 3.2.$

Pertanyaannya bukan:

"Apakah Sharpe 3.2?"

Tetapi:

"Berapa banyak strategi yang dicoba sebelum mendapatkan 3.2?"

Ini adalah pertanyaan yang jauh lebih penting.

AE.40. Deflated Sharpe Ratio

Deflated Sharpe Ratio mencoba mengoreksi interpretasi Sharpe terhadap:

- multiple testing,
- non-normality,
- selection bias.

Secara konseptual:

$$DSR = P(SR_{observed} > SR_{benchmark}).$$

Dengan benchmark yang mempertimbangkan jumlah trial dan statistik return.

Kita tidak boleh menganggap:

$SR = 2$



sebagai bukti kuat jika kita telah mencoba ribuan model.

AE.41. Probabilistic Sharpe Ratio

PSR menjawab:

Seberapa besar probabilitas bahwa true Sharpe lebih besar dari threshold tertentu?

Secara sederhana:

$$P(SR(SR^*) = P(SR_{true} > SR^*).$$

Ini lebih informatif daripada sekadar:

$$SR_{observed} = 1.8.$$

AE.42. Sharpe saja juga tidak cukup

Kita akan menyimpan minimal:

$$\{Return, Sharpe, Sortino, MaxDD, Calmar, WinRate, EV, Brier, LogLoss, Calibration\}.$$

AE.43. Max Drawdown

Equity:

$$E_t.$$

Running maximum:

$$M_t = \max_{s \leq t} E_s.$$

Drawdown:

$$DD_t = \frac{E_t - M_t}{M_t}.$$

Maximum drawdown:

$$MDD = \min_t DD_t.$$

AE.44. Mengapa drawdown penting untuk \$1 risk?

Walaupun setiap trade:

$$Risk = \$1,$$

kita masih dapat mengalami:

$$10, 20, 30, \dots$$

loss berturut-turut.

Jadi fixed-dollar risk tidak berarti fixed portfolio risk.

AE.45. Risk of ruin

Jika probabilitas loss:

$$q = 1 - p,$$

dan kita memiliki capital:

$$C.$$

Dengan risk per trade:

$$r.$$

Maka jumlah unit risk:

$$N_R = \frac{C}{r}.$$

Misalnya:

$$C = \$1000, \quad r = \$1.$$



Kita memiliki:

$$1000R.$$

Ini memberikan ruang drawdown yang jauh lebih besar daripada jika:

$$r = \$20.$$

AE.46. Tetapi \$1 risk tetap harus adaptif

Risk budget dapat ditulis:

$$R_t = R_{base} \cdot g(\text{Drawdown}_t, \text{Volatility}_t, \text{Regime}_t, \text{ModelConfidence}_t).$$

Misalnya ketika drawdown tinggi:

$$g < 1.$$

Jadi risk diturunkan.

AE.47. Model confidence

Kita dapat mendefinisikan confidence berdasarkan:

$$p_{cal},$$

uncertainty:

$$CI(p),$$

dan calibration error.

Misalnya:

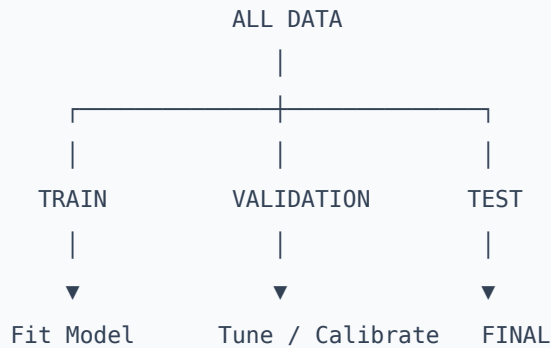
$$\text{Confidence}_t = h(p_t, CI_t, \text{CalibrationError}_t).$$

Ini lebih baik daripada:

$$\text{confidence} = |p - 0.5|.$$

AE.48. Out-of-sample hierarchy

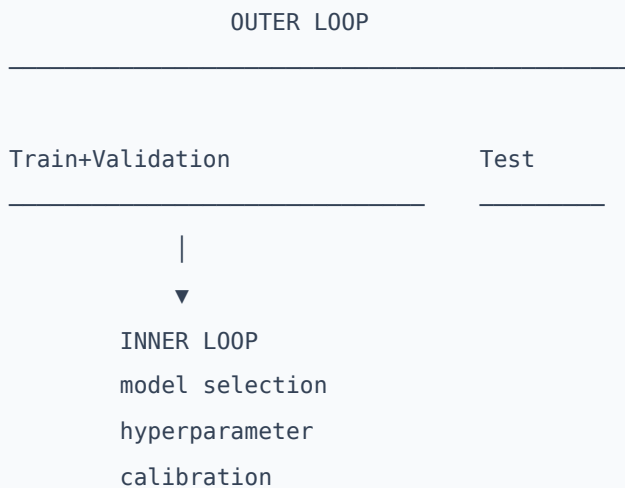
Saya menyarankan struktur dataset kita menjadi:



Tetapi untuk time series, pembagian ini dilakukan **secara temporal**, bukan random.

AE.49. Bahkan lebih baik: nested walk-forward

Misalnya:



Dengan demikian final test benar-benar untouched.

AE.50. Contoh konkret

Misalkan data:

2021 – 01 – 01 → 2025 – 12 – 31.

Kita dapat menggunakan:

Fold 1

Train:

2021

Validation:

2022

Test:

2023Q1.

Fold 2

Train:

2021 – 2022

Validation:

2023Q1 – Q2

Test:

2023Q3.

Dan seterusnya.

Dalam implementasi sebenarnya, panjang window akan disesuaikan dengan horizon dan jumlah data.

AE.51. Feature leakage

Ini salah satu jebakan terbesar.

Misalkan kita menghitung:

$$\mu_{30d}$$

menggunakan seluruh bulan.

Jika trade terjadi pada:

15 Januari,

tetapi rolling mean menggunakan data sampai:

31 Januari,

maka kita telah memasukkan masa depan.

AE.52. Rule sederhana

Untuk setiap timestamp t :

$$Feature_t = f(Data_{\leq t}).$$

Bukan:

$$f(Data_{\leq t+\Delta}).$$

AE.53. Normalization juga dapat menyebabkan leakage

Misalkan:

$$z_t = \frac{x_t - \mu}{\sigma}$$

dengan μ, σ dihitung menggunakan **seluruh dataset**.

Maka informasi future masuk ke normalization.

Seharusnya:

$$\mu_t, \sigma_t$$

dihitung hanya dari training history yang tersedia pada waktu tersebut.

AE.54. Bahkan PCA bisa leakage

Misalkan kita melakukan PCA terhadap covariance matrix:

$$\Sigma$$

menggunakan seluruh data 2021–2025.

Kemudian menggunakan principal components untuk memprediksi 2022.

Itu leakage.

PCA harus di-fit hanya pada:

$$Train_t.$$

Kemudian transform:

$$Validation_t, Test_t$$

tanpa melakukan fitting ulang.

AE.55. Factor model juga harus temporal

Misalkan:

$$R_i = \alpha_i + \beta_i^\top F + \epsilon_i.$$

Parameter:

$$\hat{\beta}$$

harus diperoleh dari data yang tersedia sebelum timestamp prediction.

Bukan dari seluruh periode.

AE.56. Survivorship bias

Ini khususnya penting untuk saham internasional.

Misalkan kita mengambil:

"100 saham terbesar saat ini."

Lalu backtest ke tahun 2021.

Masalahnya:

perusahaan yang sekarang besar sudah diketahui **ex post**.

Perusahaan yang bangkrut/delisted mungkin hilang dari dataset.

Ini menghasilkan:

Survivorship Bias.

AE.57. Universe harus point-in-time

Idealnya universe pada waktu t :

$$U_t$$

harus mencerminkan aset yang benar-benar dapat dipilih investor pada saat itu.

Jadi:

$$U_{2021} \equiv U_{2026}.$$

Ini sangat penting ketika nanti sistem kita berpindah dari crypto ke:

- US equities,
- European equities,
- Asian equities.

AE.58. Corporate actions

Untuk saham:

- stock split,
- dividend,
- merger,
- delisting.

harus diperhitungkan.

Kalau tidak:

$$Return_t$$

bisa salah.

AE.59. Crypto juga punya masalah tersendiri

Untuk crypto:

- token listing,
- delisting,
- perubahan market structure,
- exchange migration,
- perpetual funding,
- liquidity fragmentation.

Karena itu:

<i>Exchange data</i>

juga harus point-in-time.

AE.60. Backtest tidak boleh menggunakan candle close sebagai fill sempurna

Misalkan candle:

$$Open = 100$$

$$High = 105$$

$$Low = 95$$

$$Close = 103.$$

Jika kita memiliki:

$$TP = 104$$

dan:

$$SL = 96,$$

kita tahu keduanya berada dalam candle.

Tetapi kita tidak tahu:

$$TP \rightarrow SL$$

atau:

$$SL \rightarrow TP.$$

Intrabar ordering harus diketahui atau dimodelkan secara konservatif.

AE.61. Ini sangat penting untuk first-passage model

Karena kita justru mendefinisikan:

$$\tau = \min(\tau_{TP}, \tau_{SL}).$$

Jika data hanya OHLC, kita mungkin tidak mengetahui:

$$\tau_{TP} < \tau_{SL}?$$

Maka backtest harus:

- menggunakan data intrabar yang cukup granular, atau
- menggunakan asumsi konservatif.

AE.62. Execution realism

P&L backtest:

$$P nL_{ideal}$$

harus diganti menjadi:

$$PnL_{realistic} = PnL_{gross} - Fee - Spread - Slippage - Impact - Funding.$$

AE.63. Slippage model

Salah satu model sederhana:

$$Slippage = \eta \frac{Q}{Depth}.$$

Atau:

$$P_{execution} = P_{mid} + \lambda \left(\frac{Q}{D} \right)^\alpha.$$

Dengan:

- Q : order size,
- D : available depth,
- λ : market impact coefficient.

AE.64. Menguji robustness terhadap biaya

Kita tidak hanya menggunakan:

$$Cost = 0.$$

Kita harus melakukan stress test:

$$C, 1.5C, 2C, 3C.$$

Jika strategy hanya profitable pada:

$$C = 0.1bps$$

tetapi langsung hancur pada:

$$C = 1bps,$$

maka edge-nya sangat rapuh.

AE.65. Sensitivity analysis

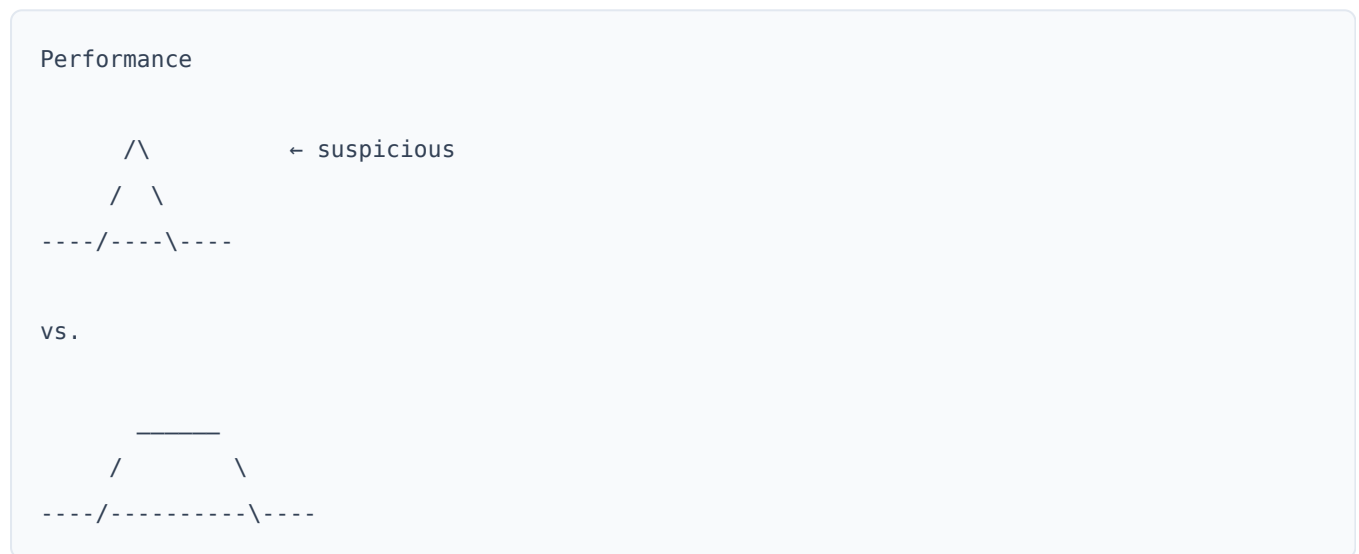
Untuk parameter:

$$\theta = (\text{lookback}, SL, TP, \text{threshold}, \text{risk}, \dots)$$

kita hitung:

$$Performance(\theta).$$

Strategi yang sehat biasanya memiliki **performance basin**, bukan satu titik tajam.



Puncak yang sangat tajam dapat menunjukkan overfitting.

AE.66. Parameter stability

Misalnya:

$$TP = 2.03R$$

menghasilkan Sharpe:

$$2.5.$$

Tetapi:

$$TP = 2.02R$$

menghasilkan:

0.3.

Ini mencurigakan.

Lebih baik jika:

$$TP \in [1.8R, 2.4R]$$

semuanya masih reasonably profitable.

AE.67. Regime robustness

Kita juga harus membagi hasil berdasarkan regime:

Bull, Bear, Sideways, HighVol, LowVol.

Misalnya:

Regime	Sharpe	EV
Bull	1.8	+
Bear	0.9	+
Sideways	0.2	~0
High Vol	1.4	+
Low Vol	1.1	+

Ini jauh lebih informatif daripada:

$$Sharpe = 1.4.$$

AE.68. Cross-asset robustness

Karena tujuan kita bukan hanya crypto, kita harus menguji:

Crypto

vs.

Equity

vs.

ETF

vs.

FX

dan seterusnya.

Tetapi **jangan memaksa satu model memiliki parameter identik untuk semuanya.**

Yang sebaiknya universal adalah:

architecture.

Bukan selalu parameter.

AE.69. Statistical validation pipeline

Maka AE akhirnya memiliki pipeline:

Historical Data

|



Point-in-Time Dataset

|



Feature Generation

|



Purged Walk-Forward Split

|



Model Training

|



Probability Calibration

|



Out-of-Sample Prediction

|



Triple-Barrier Outcome

|



Brier / LogLoss / Calibration

|



EV / PnL / Sharpe / Drawdown

|



Bootstrap / Confidence Interval

|



Multiple-Testing Correction

|



Robustness Tests

|



FINAL ACCEPTANCE

AE.70. Final acceptance criteria

Saya menyarankan kita **tidak** mendefinisikan:

"Sharpe > 1 berarti strategi diterima."

Terlalu sederhana.

Sebaliknya, strategy candidate harus melewati beberapa layer.

Statistical

Calibration acceptable

Brier acceptable

LogLoss acceptable

Economic

$EV > 0$

$Net\ PnL > 0$

MDD acceptable

Robustness

Cost stress test

Parameter stability

Regime stability

Statistical significance

PSR

dan confidence interval yang masuk akal.

AE.71. Decision gate

Kita akhirnya dapat membuat:

$$\text{TradeAllowed}_t = I(C_t > C_{min})I(EV_t > 0)I(Risk_t < R_{max})I(Liquidity_t > L_{min})$$

Tetapi bahkan ini belum cukup.

Untuk live deployment kita dapat menambahkan:

$$I(\text{ModelHealth}_t = 1).$$

Sehingga:

$$\text{TradeAllowed}_t = I(\text{Calibration} \wedge EV \wedge Risk \wedge Liquidity \wedge \text{ModelHealth}).$$

AE.72. Model health

Model dapat mengalami degradation.

Misalnya:

Training:

$$\text{Brier} = 0.12.$$

Live:

$$\text{Brier} = 0.25.$$

Maka:

$$\text{ModelHealth} \downarrow .$$

Kita tidak boleh terus meningkatkan position size hanya karena historical backtest bagus.

AE.73. Population Stability

Kita juga harus memeriksa apakah distribusi feature berubah.

Misalnya:

$$X_{train}$$

memiliki distribusi:

$$P_{train}(X).$$

Sedangkan live:

$$P_{live}(X).$$

Jika:

$$P_{live}(X) \not\equiv P_{train}(X),$$

maka terjadi:

$$Distribution\ Shift.$$

AE.74. Concept drift

Lebih buruk lagi:

$$P(Y | X)$$

sendiri dapat berubah.

Misalnya dulu:

$$OBI > 0.3$$

berarti bullish.

Tetapi setelah market structure berubah:

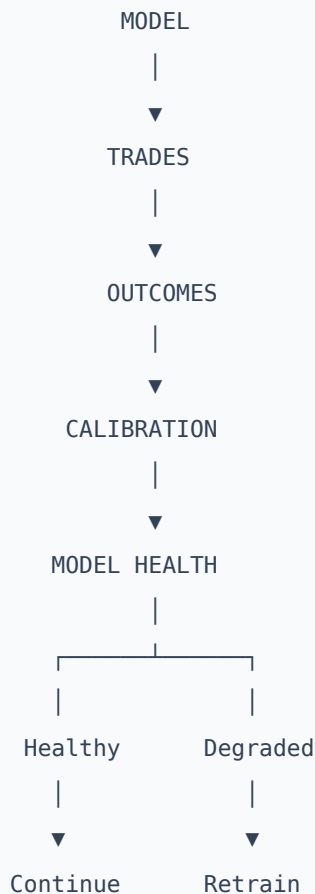
$$OBI > 0.3$$

tidak lagi memiliki predictive power yang sama.

Ini:

Concept Drift.

AE.75. Maka sistem kita harus memiliki feedback loop



Ini membuat sistem **adaptif tetapi tetap terkontrol**.

AE.76. Dan ini mengubah definisi backtest

Backtest kita bukan:

"Jalankan strategy pada data lalu lihat profit."

Tetapi:

$$\text{Backtest} = \text{Simulation} + \text{Statistical Validation} + \text{Execution Modeling} + \text{Uncertainty Quantification}.$$

AE.77. Hierarki bukti

Saya ingin kita menggunakan hierarki berikut:

$$\text{Backtest} < \text{WalkForward} < \text{OutOfSample} < \text{PaperTrading} < \text{LiveSmallCapital} < \text{ScaledLive}.$$

Semakin ke kanan, semakin kuat bukti bahwa sistem benar-benar bekerja di dunia nyata.

Tidak ada backtest yang bisa membuktikan strategy akan selalu profitable.

AE.78. Hubungan dengan quantum optimization kita

Ini juga penting untuk tujuan besar kita.

Quantum optimizer:

$$QAOA/VQE$$

tidak boleh diberi tugas:

"cari parameter yang membuat backtest paling profitable."

Itu akan sangat mudah menghasilkan overfitting.

Sebaliknya:

$$\text{Statistical validation}$$

harus berada **di luar optimizer**.

Quantum/classical optimizer hanya menyelesaikan:

$$\max_x J(x \mid \text{validated estimates}).$$

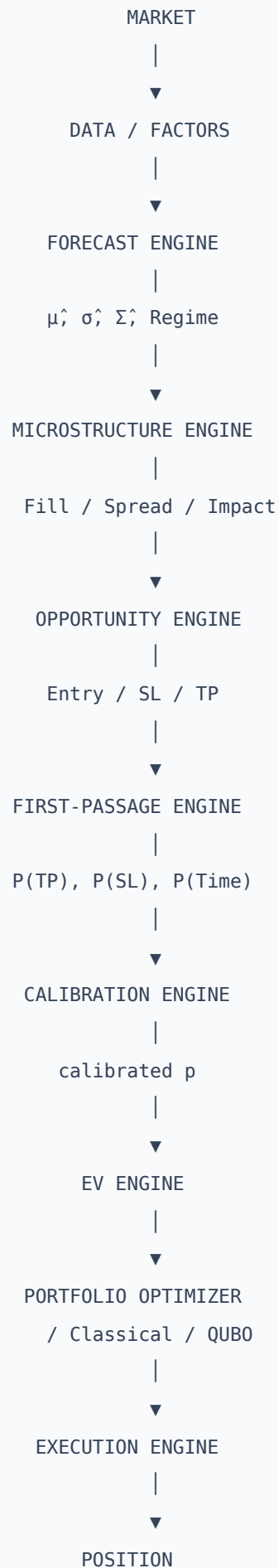
Validasi menentukan apakah:

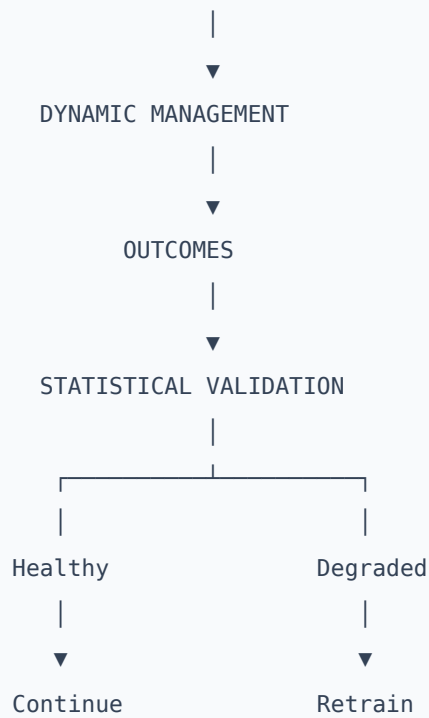
$$J$$

layak dipercaya.

AE.79. Arsitektur final sampai Tahap AE

Kita sekarang memiliki:





AE.80. Persamaan "kontrak" antar-engine

Ini menurut saya merupakan salah satu hasil paling penting dari Tahap AE.

Forecast engine memberikan:

$$(\hat{\mu}, \hat{\sigma}, \hat{\Sigma})$$

Opportunity engine mengubahnya menjadi:

$$\hat{\mathbf{p}} = (\hat{p}_{TP}, \hat{p}_{SL}, \hat{p}_{TS})$$

Calibration engine menghasilkan:

$$\tilde{\mathbf{p}} = \text{Calibration}(\hat{\mathbf{p}})$$

EV engine:

$$EV(\theta) = \tilde{p}_{TP} G - \tilde{p}_{SL} L - C$$

Optimizer:

$$\theta^* = \arg \max_{\theta} EV(\theta)$$

dan validator kemudian menguji apakah:

$$EV(\theta^*)$$

benar-benar terealisasi secara out-of-sample.

AE.81. Prinsip yang harus kita pegang

Ada satu prinsip yang menurut saya harus menjadi **aturan fundamental proyek ini**:

Jangan mengoptimalkan sesuatu yang belum tervalidasi.

Dan bahkan lebih spesifik:

Prediction → Calibration → Validation → Optimization → Execution.

Bukan:

Data → Optimization → Backtest bagus → Cari pembenaran.

Perbedaan tersebut sangat besar.

AE.82. Posisi kita sekarang

Sampai Tahap AD kita sudah menjawab:

"Bagaimana menentukan trade terbaik?"

Tahap AE menjawab:

"Bagaimana memastikan bahwa probabilitas dan edge yang digunakan untuk menentukan trade tersebut memang memiliki dasar statistik yang dapat dipercaya?"

Dengan demikian kita sudah memiliki dua komponen:

Decision Engine

dan:

Evidence Engine.

Dan **Evidence Engine** inilah yang mencegah seluruh sistem berubah menjadi overfit backtest yang sangat kompleks.

AE.83. Tahap berikutnya: AF

Setelah ini kita sudah bisa masuk ke bagian yang jauh lebih menarik: **AF — Portfolio-Level Trade Selection & Cross-Asset Risk Optimization**.

Di sana masalahnya berubah dari:

"Apakah trade A bagus?"

menjadi:

"Dari 500 trade opportunity yang muncul bersamaan, kombinasi mana yang sebaiknya kita ambil?"

Karena misalnya pada pukul 10:00 sistem menemukan:

20 crypto trades + 100 US stocks + 30 ETFs.

Masing-masing mungkin memiliki:

$$EV > 0.$$

Tetapi **kita tidak boleh mengambil semuanya**, karena exposure-nya bisa sangat berkorelasi.

Di sinilah:

$$\Sigma,$$

factor exposure,

$$VaR/CVaR,$$

drawdown constraint,

dan akhirnya **QUBO/Ising/QAOA** yang sejak awal menjadi tujuan kita mulai benar-benar bertemu dengan **short-term quantitative trading** secara natural.